Malware Math

Luc Alexander Lüthi

March 27th, 2026

Contents

1 Introduction **1**

2 Formulation **1**

3 Simple Combinators **2**

4 Immunity **2**

5 Malware and Virality **3**
5.1 Intrusion 3
5.2 Escalation 3
5.3 Persistence 3
5.4 Spread 3

1 Introduction

The claim mind model, while a decent conceptual analog, still retained a few key flaws. Defense collapses to pruning for minimal graphs, offense collapses to asserting end goal states without nuance, malware representations could not operate autonomously or replicate, and capabilities were external to the system. This note introduces a system which is closed under its own semantics, but may optionally introduce side effects based on the implementation of the language. I introduce its formal definition, some example combinators, and how it fits in to modeling adversarial systems more deeply than previous systems.

2 Formulation

As a concatenative term rewriting system, a program is constructed as a series of appensions to the program text. Writing the program and giving in put are the same action. Each appension comprises an instruction which can refer to any defined term in the sequence. I am hesitant to call the symbols which represent these instructions atoms, as they are not truly atomic and can comprise their own subprograms. Everything in this system is a concatenative program.

$$A\ B\ C\ D\ E$$

A term is constructed from symbols to describe a rewrite rule. Terms have trigger rules, which may include any number of arguments delimited by a period '.', and a rewrite, delimited by a color ':'.

```
[ .x .y swp : y x ] 1 2 swp
```

$$\Rightarrow [\ .x\ .y\ \text{swp} : \text{y x}\ ]\ 2\ 1$$

Terms are not immune from triggering rewrites on their own program text, but do not effect themselves. Appended symbols may be their own entire programs. These programs may be appended to, causing appension logics to occur of the same nature as the top level computation logic,

$$() \oplus a \oplus b$$

$$\Rightarrow (a\, b)$$

or terminated, lifting the state of the program one level higher.

$$(a\, b\, c)\ \bot$$

$$a\, b\, c$$

These programs can be expressed literally, or may be part of an instruction.

$$A \oplus a$$

$$\Rightarrow A/(a)$$

In the case of duplicate term triggers, the right most definition takes precedence. Rewrites of the program text are evaluated left to right.

3 Simple Combinators

Pulling from concatenative languages, we can manipulate our program text at a basic level with the following combinators. These correspond to common stack manipulation operations: rot, ovr, swp, dup, cut, and pop respectively.

$$[\ .x\ .y\ .z\ \rho : \text{y x z}]$$

$$[\ .x\ .y\ \omega : \text{x y x}]$$

$$[\ .x\ .y\ \sigma : \text{y x}]$$

$$[\ .x\ \delta : \text{x x}]$$

$$[\ .x\ .y\ c : \text{y}]$$

$$[\ .x\ \pi :\ ]$$

4 Immunity

Defense in this model still involves preserving capabilities, these capabilities now being represented by terms within the program, and doing so though writing term rules which prevent harmful capabilities from being generated as rules, and disallowing potentially harmful patterns from introducing malware or disabling terms in the immune system. For modeling effectful systems requiring security, we extend the model to include capabilities and instructions which have external effects and cannot be generated by the system itself. These capabilities enable us to metabolize useful signals into effects, staging the need to take, and filter input from the environment, as well as giving input back to the environment. The shape of each set of defenses depends largely on the constraints which generated them, and can vary wildly. While a blanket "defend against everything" strategy of filtering is theoretically possible, constraints prevent

this from happening in the real world. This system, in contrast to the last, actually requires its malware to be dependent on the defense of the target system. This was also the case with the graph structural capability engine.

The environment for this modeling still requires an environment of entities, each with their own mind as a program. Each entity emits signasl into the environment, and receives signals by appending them to their program and metabolizing them through rewrite rules.

5 Malware and Virality

Malware occurs when a rewrite rule is successfully generated adversarially in a program stream against the wishes of the immune system. Malware in this model can finally be truly viral and self replicating, and can even attack the immune system and escalate its influence over the system. There are a number of patterns of occurence for each stage of infection: intrusion, escalation, persistence, and spread.

5.1 Intrusion

There are three intrusion techniques I've discovered so far. The first requires passing some innocuous data paired later with a seemingly innocent rewrite rule targetting the data segment. The data segment then transforms to a malliscious rewrite rule. The second involves passing a nested group which through either the machinations of the environment or the host machine is lifted into global scope, releasing a malliscious rewrite rule. The third is the simplest, simply pass a malliscious rewrite rule through the filter.

5.2 Escalation

Depending on the nature of the initial compromise, escalation can be met typically by sending additional signals to the host, triggering more detrimental rewrite rules, potentially targetting deeper into the structure of the host or its immune system. If the initial compromise was severe enough, it may be enough to achieve escalation and persistence all in one move.

5.3 Persistence

Achieving persistence requires no self directed mutations to the hosts ruleset can disable your progress. This is most directly achievable by removing the capability responsible for signaling form the host to itself, but can also involve ensuring that if any new signals are passed, that instead your filter is dominant and does not let a more sophisticated immune system to grow. You are in this situation taking the place of the immune system, tricking the host into rejecting help as a foreign body.

5.4 Spread

Spread refers to two things. The first is a form of persistence wherein redundancy is made to ensure your rules cannot be simply purged. Textual redundancy is not typically enough, but rather semantic redundancy, otherwise a single defensive rewrite rule can render all your redundancy mute. Spread can however refer to controlling the signaling system of the host to send mallicious signals to other hosts in the environment.